LiveNoteAI: AI-Driven Meeting Transcription, Summarization and Action Item Extraction

Sai Satwik Yarapothini
Master's in Applied Data Science
Department of Engineering Education
University of Florida
saisatwi.yarapot@ufl.edu

***Abstract*—Professional meetings continuously generate high-value information such as action items, decisions and risks, yet these outcomes were often captured incompletely through manual note-taking or only summarized after the meeting by cloud-based tools that relied on third-party infrastructure. LiveNoteAI addressed this problem through a fully open-source, dual-cadence meeting intelligence pipeline that combined faster-whisper small in int8 mode for Automatic speech recognition(ASR) every 15 seconds, pyannote.audio 3.1 for speaker diarization, and a locally served Mistral 7B large language model operating every 60 seconds over sliding transcript windows to produce a running summary, action items, decisions, and risks. Before any generated output reached the user interface, a 7-rule trust validation layer enforced JSON validity, evidence grounding, owner consistency, deadline normalization, deduplication and human-lock protection. The final system achieved a word error rate of 0.230 with faster-whisper small compared with 0.276 for faster-whisper base, representing a 16.7% relative improvement, and the selected Mistral 7B extractor achieved a ROUGE-1 score of 0.072 compared with 0.029 for Llama 3.1 8B, representing a 148% improvement on the evaluation set. In addition, the trust validator intercepted 48.4% of raw large language model-generated action items as hallucinated or invalid before display, while the action item extraction pipeline achieved an LLM-judge correctness score of 3.29 out of 5. These results showed that real-time meeting intelligence was achievable on commodity hardware without mandatory cloud audio processing, and that a structured trust layer was essential because a substantial portion of raw generative output was not reliable enough for direct presentation.**

***Index Terms*—meeting intelligence, automatic speech recognition, meeting summarization, action item extraction, speaker diarization, large language models, human-in-the-loop AI**

I. Problem Statement

Meetings remain a primary mechanism for decision-making and task assignment in many organizations, yet their outcomes are frequently captured poorly or lost entirely. In practice, participants rely on manual note-taking, which divides attention between listening and documentation, leading to incomplete, inconsistent or unstructured records. Existing tools such as Zoom AI Companion, Otter.ai and Fireflies.ai partially addressed this problem through transcription and post-meeting summaries and they offer live summaries only through Paid plans which becomes the bottleneck for regular users in live meetings [13], [14], [15], but they mostly operate as cloud-based systems and emphasized after-the-fact documentation rather than real-time, structured intelligence during the meeting. As a result, users still needed to manually verify outputs, identify missing or incorrect details, and convert them into actionable items, limiting their effectiveness in supporting reliable follow-through.

The final scope of LiveNoteAI reframed the problem from static transcription to real-time meeting intelligence: transforming live audio into continuously updated, structured outputs that capture what matters operationally. Specifically, the system was designed to address the following gaps:

- **Incomplete capture of key outcomes:** Action items, decisions, risks, owners, and deadlines were often missing or ambiguous.
- **Lack of real-time feedback:** Errors in summaries or extracted insights could not be corrected during the meeting.
- **Unstructured outputs:** Existing systems produced free-form text that required manual interpretation and restructuring.
- **Limited trust and reliability:** Raw model outputs frequently contained hallucinations or inconsistencies.
- **Privacy constraints:** Dependence on third-party cloud processing made many tools unsuitable for sensitive or confidential meetings.

The significance of this problem was both practical and technical. Poor meeting documentation resulted in missed follow-ups, duplicated work, unclear ownership and delayed execution. By enabling real-time, structured and editable outputs, LiveNoteAI reduced these inefficiencies and allowed participants to validate and correct critical meeting information during the meeting itself. Furthermore, by adopting a local-first, open-source design, the system addressed privacy and deployment constraints, making it suitable for environments where cloud-based solutions were not viable.

II. Related Work

The design of LiveNoteAI was informed by prior work across automatic speech recognition, speaker diarization and large language models for structured generation. Key contributions from these areas are summarized below:

- **Speech Recognition:** Whisper demonstrated that large-scale weakly supervised training enables robust transcription across diverse audio conditions [1]. This was

operationalized using faster-whisper, which provided a low-latency, memory-efficient implementation suitable for real-time local inference [2].

- **Speaker Diarization:** pyannote.audio introduced modular neural pipelines for speaker diarization, enabling attribution of speech segments to individual speakers [3]. This capability was important for linking extracted action items and decisions to plausible owners.
- **Large Language Models:** Mistral 7B demonstrated strong reasoning and generation performance with efficient local inference [4], while Llama 3 served as a comparative baseline for structured extraction tasks [5]. These models motivated framing meeting intelligence as constrained JSON generation rather than free-form summarization.
- **Datasets and Evaluation:** Meeting corpora such as AMI and ICSI provided multi-speaker conversational data for evaluation [6]– [10], while ROUGE and WER metrics guided summarization and transcription evaluation respectively [11], [12].

In addition, existing commercial tools such as Zoom AI Companion, Otter.ai, and Fireflies.ai provided practical baselines for meeting transcription and summarization workflows [13], [14], [15]. However, these systems primarily operate as cloud-based, post-meeting assistants and provide limited support for real-time mostly in the form of paid plans for structured, and editable outputs. In contrast, LiveNoteAI introduces:

- **Real-time structured intelligence:** Continuous extraction of summaries, action items, decisions and risks during the meeting.
- **Local-first architecture:** No mandatory cloud audio processing, enabling privacy-preserving deployment.
- **Trust validation layer:** Filtering of hallucinated or inconsistent outputs before user exposure.
- **Human-in-the-loop control:** Editable outputs with protection against unintended overwrites.

These differences position LiveNoteAI as a system focused on real-time usability, reliability and deployment transparency rather than post-hoc documentation.

III. Objectives

The final objectives of LiveNoteAI are summarized as the capabilities successfully implemented and validated in the completed system:

- Real-time speech pipeline using faster-whisper (15-second chunks) with near-real-time transcript updates.
- Speaker diarization via pyannote.audio for speaker-attributed transcripts.
- Dual-cadence pipeline: 15-second ASR windows and 60-second intelligence extraction windows.
- Incremental intelligence extraction (summary, action items, decisions, risks) using Mistral 7B.
- Trust validation layer enforcing structured, grounded, and consistent outputs before display.
- Human-in-the-loop editing with lock protection to prevent overwriting user corrections.
- Live streaming architecture using FastAPI and WebSockets for real-time frontend updates.
- Export functionality (PDF and JSON) for structured meeting outputs.
- Multi-backend LLM support (Ollama, Groq, OpenAI-compatible APIs).
- Graceful degradation modes (full pipeline, no diarization, transcript-only).
- Empirical model validation using AMI and ICSI datasets.
- Reproducible local deployment using Docker and pinned dependencies.

Several objectives evolved during development due to implementation constraints and empirical findings. Whisper was replaced with faster-whisper to achieve lower latency and memory usage for real-time execution [1], [2]. Llama 3.1 8B was replaced by Mistral 7B after evaluation showed significantly better summarization performance [4], [5]. The initial plan for long audio processing was revised to a dual-cadence pipeline to improve responsiveness. Additionally, persistent database storage was removed in favor of in-memory state with export to reduce system complexity and improve privacy. Prompting strategies were simplified to schema-constrained JSON extraction for more stable outputs, and dataset scope was narrowed to AMI and ICSI, which provided sufficient coverage for evaluation within project constraints.

IV. Final Deliverables

The final deliverable of LiveNoteAI was a self-contained, locally runnable, open-source meeting intelligence system. The package was designed for reproducible execution without model training and included all components required for real-time meeting transcription, structured intelligence extraction and user interaction.

A. Backend (APIs & Processing)

- **FastAPI backend** implementing the full speech and intelligence pipeline.
- **Core REST APIs:** `/health`, `/meetings`, `/meetings/{id}`, `/export/json` and `/export/pdf`.
- **WebSocket endpoint:** Real-time session handling, audio chunk ingestion and streaming updates.
- **Speech pipeline:** Audio conversion, faster-whisper ASR, optional diarization, alignment and noise filtering.
- **Intelligence pipeline:** Mistral 7B-based extraction (summary, action items, decisions, risks).
- **Trust validation layer:** Structured filtering for correctness, grounding and consistency.
- **State management:** Incremental meeting memory and final consolidation.

B. Frontend (User Interface)

- **React single-page application** for live meeting interaction.

- **Core UI components:** Transcript, Summary, Action Items, Decisions, Risks and Export panels.
- **Live features:** Real-time transcript updates and structured output streaming.
- **Editing capabilities:** Full CRUD operations with human-lock protection.
- **Export functionality:** One-click PDF and JSON download.
- **Session management:** WebSocket integration and automatic reconnection.

C. Evaluation & Testing

- Evaluation notebooks for ASR, diarization, summarization and extraction performance.
- Metrics: WER, ROUGE, and LLM-as-judge scoring.
- Stress testing on AMI and ICSI datasets.
- Unit test suite covering trust validation, session handling and data integrity.

D. Infrastructure & Deployment

- Docker and `docker-compose` for one-command local deployment.
- Reproducible environment with pinned dependencies.
- Multi-LLM backend support (Ollama, Groq, OpenAI-compatible APIs).
- Runtime integration of pre-trained models (faster-whisper, pyannote, Mistral 7B).

E. Final Software Package

The final product was a complete local meeting intelligence system rather than a standalone trained model. It consisted of a FastAPI backend, a React frontend, integrated pre-trained models, evaluation artifacts, testing utilities and containerized deployment files. The system could be cloned and executed locally on standard hardware to provide real-time transcription, structured meeting intelligence, and human-in-the-loop interaction without requiring external cloud infrastructure.

V. Methodology

This section describes the final data pipeline, preprocessing strategy, feature engineering process and system architecture used in LiveNoteAI. LiveNoteAI methodology was organized in two parts: Dataset construction and evaluation pipeline and End-to-end meeting intelligence architecture as shown in Fig. 1.

A. Data Pipeline

The final evaluation pipeline used meeting-domain datasets selected to benchmark transcription quality, summarization quality, action-item extraction and real-time latency under realistic multi-speaker conditions.

1) Evaluation Datasets: The final evaluation pipeline used publicly available meeting-domain corpora selected to benchmark transcription quality, summarization quality, action-item extraction, and long-meeting runtime behavior. The complete dataset configuration used for model selection and final testing is summarized in Table I.

AMI was used as the primary benchmark because it provides aligned transcripts and meeting summaries, while ICSI was used to evaluate robustness on longer and less structured conversational sessions.

2) Cleaning and Preprocessing: Incoming browser audio and benchmark datasets were standardized through the following pipeline:

- Audio converted to 16 kHz mono WAV format.
- Streaming segmentation into 15-second chunks.
- Removal of empty, ultra-short, or corrupted segments.
- Low-confidence transcript filtering using ASR confidence scores.
- Text normalization for punctuation and whitespace consistency.
- Timestamp alignment to absolute meeting time.

3) Feature Engineering: Two feature streams were created for downstream intelligence extraction:

- **Speech Features:** 80-bin log-mel spectrogram inputs used by Whisper-based ASR.
- **Conversation Features:** speaker labels, timestamps, utterance duration, confidence score, and rolling transcript windows.
- **Context Features:** previous summary state, existing action items, decisions, and known speakers.

B. System Architecture

After preprocessing, LiveNoteAI converted live meeting audio into structured outputs through the five-stage architecture shown in Fig. 1.

1) Stage 1: Live Audio Capture:

- Browser `MediaRecorder` captured microphone audio.
- Audio encoded in WebM/Opus format.
- Chunks transmitted through persistent WebSocket connection.

2) Stage 2: Speech-to-Text Layer: The ASR module used faster-whisper [1], [2].

Final Hyperparameters:

- Model: `whisper-small`
- Compute Type: `int8`
- Beam Size: 5
- Device: CPU
- VAD Filter: Enabled

This configuration achieved WER = 0.230 and outperformed the base model (0.276). Speaker diarization was implemented using pyannote.audio [3]. Transcript segments were matched to speaker turns using maximum-overlap alignment.

TABLE I
FINAL DATASETS USED FOR EVALUATION AND MODEL SELECTION.

Dataset	Subset Used	Size	Purpose
AMI Meeting Corpus (`edinburghcstr/ami`, IHM) [6], [9]	Test split	250 utterances (WER); 25 meetings (ROUGE); 4 meetings / 31 action items (LLM judge)	Primary benchmark for ASR accuracy, summarization quality, and structured action-item extraction
ICSI Meeting Corpus (`argmaxinc/icsi-meetings`) [8], [10]	Test split	5 full meetings (∼57 min average, 20+ chunks each)	Long-form latency stress testing under realistic multi-speaker meeting conditions

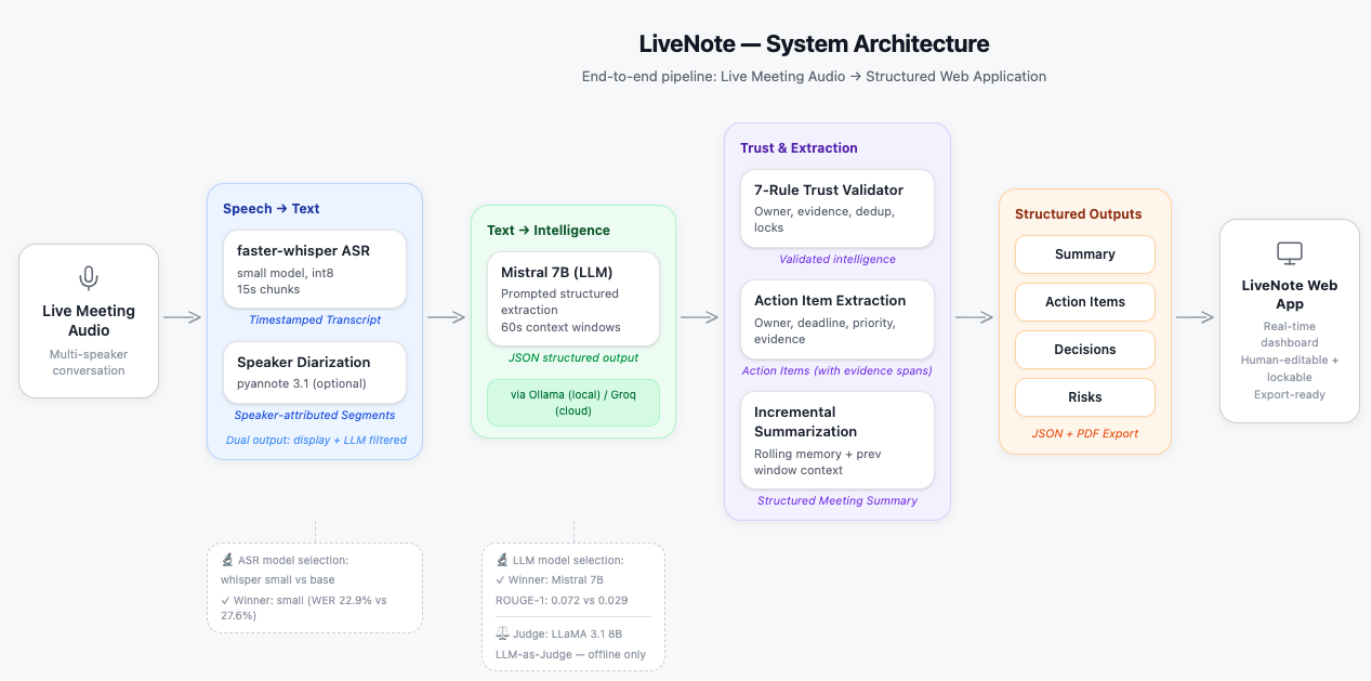

Fig. 1. Final end-to-end architecture of LiveNoteAI. Live audio was processed through Automatic speech recognition, Speaker diarization, structured intelligence extraction, trust validation and real-time web delivery.

3) Stage 3: Intelligence Layer: The structured reasoning layer used Mistral 7B [4] as the final production model, while Llama 3.1 8B was retained as a comparison baseline [5].

Inference Configuration:

- Temperature: 0.1
- JSON-only output mode
- 60-second rolling transcript windows
- Token budget: ∼1500–2000

The model generated:

- Running summaries
- Action items
- Decisions
- Risks

4) Stage 4: Trust Validation Layer: A deterministic 7-rule trust-validation layer was placed between raw LLM output and the persistent meeting state. This layer was designed to ensure that generated content was not surfaced directly to the user without structural, semantic and human-control checks. In practice, it served as a reliability boundary between probabilistic generation and user-facing meeting records.

- **Rule 1: Owner Validation** — assigned owners had to belong to the known speaker set extracted from the transcript or diarization pipeline. If no valid owner match existed, the owner field was replaced with `unassigned` and the item was flagged for review.
- **Rule 2: Evidence Span Validation** — every extracted action item, decision, or risk was required to contain valid supporting evidence spans grounded in the current transcript window. Out-of-range, malformed, or missing spans caused the item to be removed or marked as invalid.
- **Rule 3: Deadline Normalization** — natural-language deadlines such as "tomorrow," "next week," or calendar dates were normalized into a structured representation. If a deadline could not be resolved reliably, it was replaced with `unspecified` to avoid presenting false precision.
- **Rule 4: JSON Validity** — all model outputs were required to conform to the expected JSON schema. Malformed JSON, schema violations, or missing required fields caused the response to be discarded before state update.
- **Rule 5: Deduplication** — repeated or semantically over-

lapping action items were removed to prevent the same task from appearing multiple times across incremental extraction windows.

- **Rule 6: Human-Lock Protection** — any summary field or structured item that had been manually edited and locked by the user was protected from overwrite by subsequent model generations, including both incremental updates and final consolidation.
- **Rule 7: Minimum Content Quality** — outputs that were empty, excessively vague, unsupported by transcript evidence, or too weak for reliable presentation were filtered before display.

This validation layer was a core methodological component rather than a minor post-processing step. It improved output reliability, reduced hallucinated or weak extractions, preserved user trust, and ensured that the final meeting state remained evidence-grounded and human-controllable.

5) Stage 5: Output Delivery: Validated outputs were streamed to the React frontend in real time. Users could edit and lock any generated field. Final meeting records were exportable in JSON and PDF formats.

The final methodology combined streaming speech recognition, rolling-context intelligence extraction, deterministic trust filtering and human-in-the-loop correction into a deployable local-first meeting intelligence system.

VI. System Design

LiveNoteAI followed a three-layer system design consisting of a React frontend, a FastAPI backend, and an in-memory session-state layer. Unlike conventional SaaS architectures, the final implementation did not require an external database for core operation. Instead, active meeting state was maintained in memory through a structured `MemoryState` object and exported as JSON or PDF when required. This design reduced infrastructure complexity, minimized storage overhead and aligned with the privacy-first goals of the project. Fig. 2 presents the final end-to-end system data flow. [2], [3], [4]

Fig. 2. Final system data flow of LiveNoteAI. Browser audio was streamed to the FastAPI backend, processed through speech and intelligence pipelines, stored in rolling meeting state, and returned to the frontend as real-time structured outputs.

A. Frontend Layer

The frontend was implemented as a React single-page application using React 18, TypeScript and Vite. It served both as the presentation interface and the human-in-the-loop control surface for meeting intelligence workflows.

The user interface consisted of six core panels:

- `TranscriptPanel` for live transcript rendering with speaker labels,
- `SummaryPanel` for rolling summary updates,
- `ActionItemsPanel` for task management,
- `DecisionsPanel` for confirmed decisions,
- `RisksPanel` for identified blockers or concerns,
- `ExportPanel` for PDF and JSON downloads.

A persistent WebSocket connection to `/ws/meeting` enabled low-latency bidirectional communication. Browser-side audio capture was handled using the MediaRecorder API, while custom React hooks managed session state, transcript rendering, and reconnection behavior. Any user edit generated a structured update event that was sent to the backend and preserved through lock protection.

B. Backend Layer

The backend was implemented using FastAPI and acted as the orchestration layer for all meeting processing tasks. Its responsibilities included session creation, WebSocket handling, audio chunk routing, speech recognition, intelligence extraction, meeting-state updates, and export generation.

To support continuous streaming, the backend maintained a per-meeting asynchronous processing queue. This enabled a dual-cadence execution model:

- **15-second cadence:** audio chunks were converted, transcribed, optionally diarized, filtered, and appended to transcript state.
- **60-second cadence:** every four chunks, the intelligence pipeline was triggered to update summaries, action items, decisions, and risks.

- **Meeting end cadence:** final consolidation was executed before export generation.

This architecture separated fast transcript responsiveness from slower reasoning workloads, improving perceived latency while preserving contextual extraction quality.

C. Processing Modules

The backend consisted of two major modules:

- **Module 1: Speech Pipeline**
  - WebM to WAV conversion using `ffmpeg`
  - faster-whisper ASR for transcription [1], [2]
  - Optional pyannote.audio diarization [3]
  - Speaker-segment alignment using maximum-overlap matching
  - Dual-output transcript filtering for display and LLM usage
- **Module 2: Intelligence Pipeline**
  - Prompt construction from rolling transcript windows
  - LLM inference using Mistral 7B / interchangeable backends [4], [5]
  - 7-rule trust validation layer
  - Incremental memory merging
  - Final consolidation at meeting completion

D. State Management and Synchronization

The final system used no persistent database. Instead, `session_manager.py` stored active meetings in a Python dictionary keyed by `meeting_id`. Each session contained transcript buffers, prior reasoning windows, known speakers, generated outputs and human-lock metadata.

After every processing cycle, updates were broadcast through `websocket_manager.py`. The three primary outbound events were:

- `transcript_update`
- `intelligence_update`
- `consolidation_complete`

This ensured that all connected clients remained synchronized in near real time.

E. Deployment and Containerization

The final implementation prioritized local reproducibility over managed cloud hosting. While cloud deployment was evaluated, optional diarization and local-model execution imposed memory constraints that were impractical on free-tier platforms. The final repository therefore supported two deployment modes:

- **Docker Compose Deployment**
  - FastAPI backend container
  - Ollama sidecar container
  - Persistent model-cache volume
  - One-command startup using `docker-compose up`
- **Manual Local Deployment**
  - Frontend via Vite
  - Backend via Uvicorn
  - Local model serving via Ollama

The backend container used a lightweight `python:3.10-slim` base image with `ffmpeg` installed for audio conversion.

F. Feature-Flagged Operating Modes

To support graceful degradation under varying hardware constraints, two runtime flags were exposed: `DIARIZATION_ENABLED` and `LIVE_INTELLIGENCE_ENABLED`. These produced four operating modes:

- Full pipeline mode
- No-speaker live mode
- Transcript-plus-speaker mode
- Transcript-only mode

This allowed the same architecture to remain functional across low-resource and full-feature deployments.

VII. Experimental Design

The experimental design of LiveNoteAI evaluated whether the final system could operate accurately and efficiently under realistic local-execution constraints. All experiments were inference-only; no model training or fine-tuning was performed. The evaluation focused on four aspects: transcription quality, structured intelligence quality, runtime latency and output reliability.

A. Testing Environment

Experiments were conducted on consumer-grade local hardware without GPU acceleration. This constraint was intentional because the target deployment scenario was standard personal computers rather than specialized servers.

- **Hardware:** CPU-only local execution.
- **Software Stack:**
  - Python 3.10+
  - `faster-whisper` 1.0.3 [2]
  - `pyannote.audio` 3.1.1 [3]
  - `torch` 2.3.1
  - `jiwer` 3.0.4 [12]
  - Ollama runtime for local LLM serving
  - Mistral 7B and Llama 3.1 8B models [4], [5]
- **Execution Mode:** Reproducible notebook-based and script-based evaluation.

B. Datasets and Evaluation Splits

All experiments used publicly available meeting datasets. AMI was used for accuracy-focused evaluation, while ICSI was used for long-form latency stress testing.

The following subsets were used:

- 250 AMI utterances for ASR WER comparison.
- 25 AMI meetings for summarization evaluation.
- 20 AMI meetings for LLM model comparison.
- 4 meetings / 31 action items for judge-based extraction evaluation.
- 5 full ICSI meetings for long-duration latency testing.

C. Baselines and Comparisons

Three baseline comparisons were used to validate major design decisions:

1) **ASR Model Comparison:** faster-whisper `base` vs. `small` under identical CPU int8 settings.
2) **LLM Extractor Comparison:** Llama 3.1 8B vs. Mistral 7B using identical prompts, validation logic, and meeting inputs.
3) **Summarization Strategy Comparison:** end-of-meeting single-pass summarization vs. incremental 60-second rolling summarization.

These comparisons isolated the effect of model choice and pipeline design while keeping the rest of the system constant.

D. Evaluation Metrics

Different metrics were used for different components:

- **Speech Pipeline:**
  - Word Error Rate (WER)
  - Speaker alignment quality
  - Per-chunk latency
- **Intelligence Pipeline:**
  - ROUGE-1 / ROUGE-2 / ROUGE-L [11]
  - JSON validity rate
  - Trust-rule trigger rate
  - Judge-based action-item score

E. Measured Resources and Scope Limits

In addition to quality metrics, operational feasibility was measured through ASR chunk latency, LLM cycle latency, queue stability, and long-meeting robustness.

The experimental scope was intentionally bounded:

- No model fine-tuning or retraining.
- No custom annotated dataset creation.
- ICSI used for latency testing only.
- No broad cross-dataset generalization study.

These limits kept the evaluation focused on the final implemented use case: a local, real-time, inference-only meeting intelligence system.

VIII. Results and Analysis

The final results were organized around four core questions: (1) which ASR model best supported real-time transcription, (2) which LLM best supported structured meeting intelligence extraction, (3) whether incremental summarization degraded quality relative to single-pass summarization, and (4) how reliable the raw action-item extraction was before trust filtering. Because LiveNoteAI was a multi-stage meeting intelligence pipeline rather than a binary classifier, the most appropriate evaluation metrics were WER, ROUGE, latency, JSON validity, trust-violation rate, and judge-based action-item scores. Confusion matrices and ROC curves were not included because the evaluated tasks did not naturally reduce to threshold-based classification.

TABLE II
ASR MODEL COMPARISON ON AMI TEST UTTERANCES

Metric	base	small
Corpus WER	0.2759	0.2295
Mean utterance WER	0.5219	0.4044
Median utterance WER	0.3229	0.2450
Filtered corpus WER	0.2386	0.2020

TABLE III
LLM COMPARISON FOR STRUCTURED EXTRACTION

Metric	Llama 3.1 8B	Mistral 7B
ROUGE-1	0.0285	0.0718
ROUGE-2	0.0042	0.0156
ROUGE-L	0.0204	0.0474
JSON validity	0.90	0.90
Median latency (s)	67.4	88.4
Violations / window	3.80	8.15

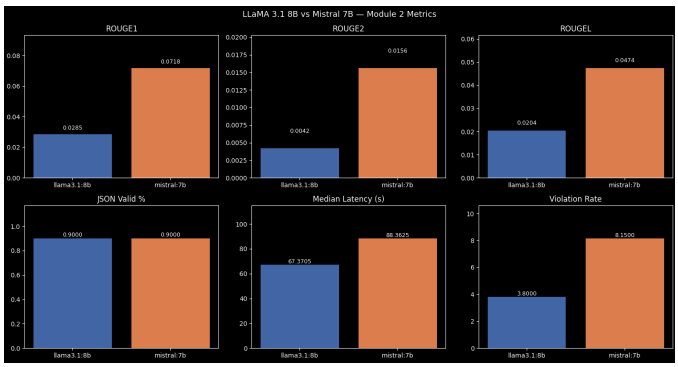

Fig. 3. Head-to-head Module 2 comparison between Llama 3.1 8B and Mistral 7B. Mistral achieved clearly stronger ROUGE scores, while Llama remained faster and produced fewer trust violations.

A. ASR Model Selection

The first experiment compared `faster-whisper base` and `faster-whisper small` on a 250-utterance subset of the AMI IHM test split under identical inference settings. Table II reports the main ASR results.

The results showed that `whisper-small` reduced corpus-level WER by 16.7% relative to `whisper-base`, while also improving filtered WER on more meaningful utterances. A paired t-test on utterance-level normalized WER produced $t \approx 2.32$ with $p \approx 0.021$, indicating that the improvement was statistically significant at $\alpha = 0.05$. This established `whisper-small` as the final ASR model.

B. LLM Extractor Comparison

The second experiment compared Llama 3.1 8B and Mistral 7B on 20 AMI meetings using identical prompts, identical trust validation, and identical memory merge logic. Only the LLM backend changed. Table III summarizes the comparison, and Fig. 3 visualizes the same results.

Mistral 7B clearly outperformed Llama 3.1 8B on all ROUGE-based summary quality metrics, with especially large gains on ROUGE-1 and ROUGE-2. JSON validity was tied at

TABLE IV
INCREMENTAL VS. SINGLE-PASS SUMMARIZATION

Metric	Single-Pass	Incremental
ROUGE-1	0.0218	0.0271
ROUGE-2	0.0037	0.0044
ROUGE-L	0.0171	0.0202
Mean latency (s)	65.87	65.92

TABLE V
JUDGE-BASED ACTION-ITEM QUALITY SUMMARY

Metric	Value
Correctness (1–5)	3.29
Specificity (1–5)	2.68
Grounding (1–5)	2.65
Hallucination flagged	48.4%

90%, so neither model had an advantage in schema compliance. Llama was faster and triggered fewer trust violations, but summary quality was the most visible output to the user. For that reason, Mistral 7B was selected as the final production extractor. A formal paired statistical test was not reported for this comparison because only aggregate ROUGE outputs were retained from the notebook; however, the relative performance gap was large enough to be practically decisive.

C. Incremental vs. Single-Pass Summarization

A central design claim of LiveNoteAI was that incremental summarization could support live updates without degrading summary quality. This was tested on 25 AMI meetings by comparing 60-second rolling summarization against a conventional end-of-meeting single-pass baseline. The quantitative comparison is reported in Table IV.

As shown in Table IV, incremental summarization improved all three ROUGE metrics in absolute terms while maintaining near-identical latency. A one-sample t-test on the per-meeting ROUGE-1 deltas yielded $t \approx 0.985$ with $p \approx 0.335$. This result was not statistically significant, which means the incremental strategy should be interpreted as achieving practical parity rather than a proven statistical improvement. That result still mattered: the live incremental design did not harm summary quality while enabling in-meeting updates that the single-pass baseline could not provide.

D. Action-Item Quality Analysis

The final evaluation examined action-item quality using a judge-based setup in which Mistral 7B served as the extractor and Llama 3.1 8B served as the judge on 31 extracted items. Table V reports the key results.

The action-item results were mixed. Correctness was moderate, but specificity and grounding were weaker, and nearly half of the raw extracted items were flagged as hallucinated or weakly supported. This result should not be interpreted as the same error rate reaching the final user interface. Instead, it justified the inclusion of the trust-validation layer, whose role was precisely to block weak, unsupported, or inconsistent outputs before display.

E. Key Findings

The final results support three main conclusions:

- **ASR choice:** `whisper-small` was the correct transcription model because it achieved significantly lower WER than `whisper-base` while remaining compatible with local real-time execution.
- **LLM choice:** Mistral 7B was the correct extraction model because it produced much stronger summary quality than Llama 3.1 8B, despite being slower and noisier before trust filtering.
- **Incremental design:** incremental summarization achieved practical parity with single-pass summarization, which supported the core design goal of delivering live meeting intelligence without sacrificing output quality.

IX. INTERPRETABILITY AND ERROR ANALYSIS

This section examines where LiveNoteAI failed, why those failures occurred, and how the final system exposed interpretable evidence to the user. Because LiveNoteAI was a multi-stage generative pipeline rather than a fixed-label classifier, the most important errors emerged in two components: (1) Speech pipeline and (2) Structured extraction pipeline.

A. Failure Modes in the Speech Pipeline

1) Short or Low-Signal Utterances: The highest transcription errors occurred on very short acknowledgments, fragmented speech, or low-information clips such as “yeah”, “no”, or partial phrases. These segments provided limited acoustic evidence, which increased decoding ambiguity.

Typical effects included:

- insertion of extra words,
- substitution with fluent but incorrect phrases,
- unstable timestamps.

To reduce downstream impact, the production pipeline filtered low-confidence, ultra-short, and filler-only utterances before they entered the LLM reasoning stage.

2) Overlapping Speech and Speaker Errors: Speaker attribution degraded when multiple speakers talked simultaneously or when diarization boundaries did not align cleanly with ASR segments.

The main technical causes were:

- chunk-level diarization rather than full-meeting diarization,
- one-speaker max-overlap assignment for multi-speaker segments,
- missing overlap between transcript and diarization windows.

When uncertainty was high, the system explicitly assigned `SPEAKER_UNKNOWN` rather than fabricating confidence.

B. Failure Modes in the Intelligence Pipeline

1) Hallucinated Action Items: The most important Module 2 failure was generating plausible but unsupported action items. Example outputs included:

- “Add radar functionality to the product”

TABLE VI
COMMON CHARACTERISTICS OF FAILURE CASES

Condition	Typical Failure
Utterance < 1 sec	High WER / insertion errors
Overlapping speech	Wrong speaker label
Vague discussion	Low-specificity task
No clear owner mention	Misattribution
Relative time phrases	Deadline unresolved
No transcript evidence	Hallucinated item

- "Use solar power for the next prototype"
- "Create clay model for industrial testing"

These tasks sounded realistic but were not grounded in the transcript.

The technical reason was that LLMs sometimes completed likely meeting patterns from prior knowledge instead of extracting only what was explicitly said.

2) Low-Specificity Tasks: Some extracted tasks were related to the discussion but too vague to execute, such as:

- "Discuss design ideas"
- "Present marketing suggestions"

This occurred when exploratory discussion resembled commitment language. The model did not always distinguish between brainstorming and confirmed ownership.

3) Owner and Deadline Errors: Owner attribution failed when speaker identity was weak or unavailable. Deadline normalization failed most often on vague expressions such as:

- next week
- before next meeting
- soon
- end of sprint

These were converted to `unassigned` or `unspecified` and flagged for review.

C. Common Characteristics of Failure Cases

Across both modules, incorrect outputs shared common patterns. The recurring conditions and their typical downstream errors are summarized in Table VI.

As shown in Table VI, the common thread across failure cases was reduced source clarity. The less explicit the speech signal or conversational intent, the more likely the system was to produce unreliable outputs.

D. Explainability Through Evidence Spans

LiveNoteAI did not use SHAP or LIME because the final system was not a tabular classifier. Instead, interpretability was implemented through **evidence-span attribution**, which was more suitable for a generative meeting assistant.

Every extracted item contained supporting timestamps linking it back to the transcript.

```
{
 "task": "Finalize vendor shortlist",
 "owner": "SPEAKER_A",
 "evidence": [{"start":142.5,"end":156.8}]
}
```

This provided practical explainability by allowing users to inspect the exact transcript region that triggered the output.

E. Trust and Safety (Microsoft HAI Alignment)

The final design aligned with Microsoft Human-AI Interaction principles for uncertainty handling and explanation.

1) G10: Scope Services When in Doubt: When confidence was weak, the system reduced certainty instead of overstating accuracy:

- `SPEAKER_UNKNOWN` labels,
- `needs_review` flags,
- `unassigned` owners,
- `unspecified` deadlines,
- transcript-only fallback mode.

2) G11: Make Clear Why the System Did What It Did: The system exposed reasoning through:

- evidence timestamps,
- trust-rule correction logs,
- visible human-lock indicators,
- editable outputs tied to transcript evidence.

This reduced black-box behavior and improved user oversight.

LiveNoteAI did not fail randomly. Errors clustered around predictable low-signal conditions: short speech, overlap, vague commitments, missing owners, and ambiguous deadlines. More importantly, the final system surfaced these weaknesses through evidence spans, uncertainty labels, validation rules, and human review controls. This made the assistant inspectable and safer for real-world meeting use.

X. CHALLENGES AND LESSONS LEARNED

This section summarizes the main technical hurdles encountered during the development of LiveNoteAI, the solutions adopted, and the key lessons that emerged from those decisions. It also includes a compact ablation-style retrospective showing which components had the greatest effect on final system quality.

A. Technical Retrospective

The development process exposed several practical challenges beyond model selection alone. The most important hurdles and their resolutions are summarized below.

- **Diarization resource and access constraints:** The `pyannote.audio` diarization pipeline required gated Hugging Face access and imposed substantial memory overhead, which made lightweight hosted deployment impractical [3]. This was addressed by making diarization optional through the `DIARIZATION_ENABLED` feature flag and supporting transcript-only fallback operation.
- **Audio conversion reliability:** WebM-to-WAV conversion was not equally stable across environments, especially when system-level `ffmpeg` paths differed. This was resolved by implementing a dual-path conversion strategy using `pydub` first and explicit `ffmpeg` fallback second.

- **LLM JSON consistency:** Early model outputs frequently contained malformed JSON, markdown fences, or conversational text instead of valid structured output. This was improved by reducing temperature to 0.1, enforcing JSON-only prompting, and adding defensive parsing in the validator.
- **Queue backlog in live mode:** Continuous chunk uploads could cause latency drift and make transcript updates fall behind the live meeting. A bounded asynchronous queue with overflow notification was introduced so that the system preserved responsiveness under pressure.
- **Human-edit overwrite conflicts:** Early merge logic did not consistently preserve user-corrected fields across repeated model updates. This was fixed by enforcing human-lock protection as an invariant across all update paths and validating it through unit tests.
- **Misleading ASR evaluation on short clips:** Utterance-level WER on very short segments produced inflated and sometimes misleading values. Final reporting therefore emphasized corpus-level WER and filtered WER as the primary comparison metrics.
- **Long-context inference timeouts:** Longer meetings increased LLM latency and sometimes caused timeout failures in evaluation. This reinforced the decision to retain the 60-second incremental window design rather than rely on longer single-pass contexts.

B. Ablation-Style Findings

A compact ablation-style review showed which system components mattered most to final performance and reliability.

- Replacing `whisper-small` with `whisper-base` increased corpus WER from 0.230 to 0.276, confirming that ASR model choice materially affected transcription quality.
- Replacing Mistral 7B with Llama 3.1 8B reduced ROUGE-1 from 0.072 to 0.029, showing that LLM choice had the largest direct effect on summary quality [4], [5].
- Removing the trust validator would have allowed unsupported or hallucinated outputs to reach the interface directly, which confirmed that validation was a core reliability component rather than a cosmetic post-processing step.
- Removing incremental processing would have eliminated live updates while also reducing summary quality relative to the final design.
- Disabling diarization removed meaningful speaker attribution, which weakened owner identification for extracted action items.
- Removing human-lock preservation would have allowed later model generations to overwrite confirmed user edits, directly reducing trust in the interface.

C. Key Lessons Learned

The final engineering lessons were straightforward.

- Reliable meeting intelligence required strong system design controls, not just strong models.
- Structured validation was essential because raw generative outputs were not dependable enough for direct presentation.
- Incremental processing was justified both architecturally and practically because it improved usability without harming quality.
- Human override mechanisms were necessary for trust, especially in a live editable interface.
- Local-first deployment was more realistic and robust for this project than dependence on cloud-only infrastructure.

The main lesson from LiveNoteAI was that building a trustworthy real-time AI system required equal attention to modeling, orchestration, validation and user control. Several of the hardest problems were not purely algorithmic: audio conversion, queue management, JSON reliability and edit preservation all had to be solved before the system became practically usable. The final architecture therefore emerged through iterative refinement toward reliability, transparency and local deployability.

XI. Conclusion and Future Work

LiveNoteAI demonstrated that real-time meeting intelligence could be achieved on commodity CPU hardware without mandatory cloud audio processing, paid subscriptions or GPU dependence. The final system integrated live transcription, optional speaker attribution, incremental summarization and structured extraction of action items, decisions, and risks into a single local-first pipeline. More importantly, the major architectural and model-selection decisions were supported by empirical evaluation rather than assumption. The final system selected `faster-whisper small` over `whisper-base` because it reduced corpus-level WER from 0.276 to 0.230, and it selected Mistral 7B over Llama 3.1 8B because it improved ROUGE-1 from 0.029 to 0.072 under identical evaluation conditions.

The main technical contribution of LiveNoteAI was not a single model, but the integration of several design choices into a practical end-to-end system:

- a dual-cadence architecture that separated 15-second ASR updates from 60-second intelligence updates;
- a 7-rule trust validator that intercepted unsupported or malformed structured outputs before display;
- a permanent human-lock mechanism that preserved user edits across subsequent model cycles;
- a local-first deployment model that prioritized privacy and open accessibility over cloud dependence.

The practical impact of the project was that it provided a zero-cost, privacy-preserving alternative to commercial meeting assistants by surfacing structured outputs while the meeting was still happening rather than only after it ended. At the same time, the results also exposed clear limitations. The summarization scores remained low in absolute terms, raw hallucination rates were substantial before validation, and action-item specificity was inconsistent. These limitations did not invalidate the system, but they established a realistic

baseline for what small open-weight models could currently achieve on conversational meeting data without task-specific fine-tuning.

Several next steps would improve the system further.

- **Meeting-domain adaptation of the ASR model:** Fine-tuning or adapter-based adaptation of the speech model on meeting-specific corpora such as AMI and ICSI could reduce transcription errors on disfluent speech, short acknowledgments, and meeting-specific vocabulary. Since downstream extraction quality depends directly on transcript quality, ASR adaptation is one of the most promising high-leverage improvements.
- **Lower-latency streaming intelligence extraction:** The current architecture waited for a 60-second transcript window before issuing a blocking LLM call. A future version could adopt streaming LLM inference and incremental partial-state updates, reducing perceived intelligence latency from approximately one minute to only a few seconds. This would make the system feel substantially more interactive during live meetings.
- **Persistent cross-meeting memory:** The current implementation treated each meeting as an isolated session and exported final state only at the end. A future extension could persist structured meeting records into a local vector or document store, enabling cross-meeting retrieval and follow-up analysis such as tracking unresolved action items or tracing decisions across multiple meetings.

In summary, LiveNoteAI showed that trustworthy, real-time, local meeting intelligence was technically feasible, but that reliability required more than transcription and summarization alone. The final system's value came from combining open models, deterministic validation, and human oversight into a coherent workflow. Future work should therefore focus not only on improving model quality, but also on preserving the transparency, editability, and privacy guarantees that made the current system practically useful.

XII. Appendices

The final project artifacts required for reproducibility and demonstration are provided below.

- **GitHub Public Repository Link:** LiveNoteAI GitHub Repository Link
- **Video Demo of the Functional UI:** LiveNoteAI Demo Video

The GitHub repository contains the complete LiveNoteAI source code, including the backend and frontend implementations, environment configuration files, evaluation notebooks, and project documentation. The repository also includes the required `README.md` and `requirements.txt` files for setup and reproducibility.

The video demo presents the functional user interface and the end-to-end workflow of the system, including live transcription, structured intelligence updates, editable meeting outputs, and final export behavior.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," arXiv preprint arXiv:2212.04356, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[2] SYSTRAN, "faster-whisper," GitHub repository. [Online]. Available: https://github.com/SYSTRAN/faster-whisper. Accessed: Feb 1, 2026.

[3] H. Bredin, R. Yin, J. M. Coria, G. Gelly, P. Korshunov, M. Lavechin, D. Fustes, H. Titeux, W. Bouaziz, and M.-P. Gill, "pyannote.audio: Neural building blocks for speaker diarization," arXiv preprint arXiv:1911.01255, 2019. [Online]. Available: https://arxiv.org/abs/1911.01255

[4] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. Renard Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7B," arXiv preprint arXiv:2310.06825, 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[5] Llama Team, "The Llama 3 herd of models," arXiv, Jul. 2024. [Online]. Available: https://ai.meta.com/research/publications/the-llama-3-herd-of-models/

[6] Hugging Face, "edinburghcstr/ami," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/edinburghcstr/ami. Accessed: Feb 1, 2026.

[7] Hugging Face, "knkarthick/AMI," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/knkarthick/AMI. Accessed: Feb 1, 2026.

[8] Hugging Face, "argmaxinc/icsi-meetings," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/argmaxinc/icsi-meetings. Accessed: Feb 1, 2026.

[9] AMI Corpus, "Welcome to the AMI Corpus." [Online]. Available: https://groups.inf.ed.ac.uk/ami/corpus/. Accessed: Feb 1, 2026.

[10] ICSI Corpus, "Welcome to the ICSI Corpus." [Online]. Available: https://groups.inf.ed.ac.uk/ami/icsi/. Accessed: Feb 1, 2026.

[11] K. Ganesan, "ROUGE 2.0: Updated and improved measures for evaluation of summarization tasks," arXiv preprint arXiv:1803.01937, 2018. [Online]. Available: https://arxiv.org/abs/1803.01937

[12] 8x8, "jiwer," PyPI. [Online]. Available: https://pypi.org/project/jiwer/. Accessed: Feb 1, 2026.

[13] Zoom, "Zoom AI Companion." [Online]. Available: https://www.zoom.com/en/products/ai-assistant/. Accessed: Feb 1, 2026.

[14] Otter.ai, "Otter." [Online]. Available: https://otter.ai/. Accessed: Feb 1, 2026.

[15] Fireflies.ai, "Fireflies.ai." [Online]. Available: https://fireflies.ai/. Accessed: Feb 1, 2026.